Tabla de contenidos

1. ¿Qué es una guía de estilo en programación?
2. ¿Por qué usar una guía de estilo?
3. Guía de estilo en Python: PEP8

¿Qué es una guía de estilo en programación?

Una guía de estilo es un conjunto de reglas de escritura de código que un programador, o un equipo de ellos, sigue para escribir el código de un programa. Esto asegura que todo archivo escrito siga el mismo formato y parezca provenir de un mismo autor. Estas reglas puede ser variadas: desde **reglas de formato** que deciden si llamar a una variable `mivariable`, `mi_variable` o `miVariable`; hasta **reglas semánticas** que deciden si llamar a una función que suma elementos de una lista `funcion_que_suma` o simplemente `suma`. Qué y cuántas reglas se usan en un proyecto queda completamente definido a gusto de los colaboradores. En general, al trabajar en un equipo grande, se definen en conjunto estas reglas antes de comenzar el proyecto o se usan reglas previamente creadas.

¿Por qué usar una guía de estilo?

A primera vista, el seguir este tipo de reglas puede verse como una restricción o más trabajo a la hora de programar. Sin embargo, el escribir código conscientemente siguiendo una guía de estilo trae muchas ventajas. Específicamente, trae ahorros de tiempo considerables tanto a la hora de escribir código como posterior a su escritura. Es por esto que su uso es una metodología de desarrollo popular en la industria. Distintos equipos de desarrollo, en lugares como Facebook, Google, Platanus, Fintual, Cornershop y más, utilizan diversas guías de estilo, dependiendo de los proyectos y lenguajes utilizados.

A continuación se listan algunas ventajas concretas de usar guías de estilo:

1. El código es más leíble

El tener reglas de escritura de variables, funciones y clases permite que el código completo sea más fácil de leer. Al programar, uno pasa más tiempo leyendo código que escribiéndolo, por lo que es fundamental contar con código legible que nos permita entender bien o que alguien más entienda lo que ocurre en el programa. Python específicamente, tiene la gracia de una sintaxis muy legible, pero esta propiedad puede perderse rápido con malos hábitos de escritura.

2. Hace el *debuggeo* más rápido

Siguiendo la idea anterior: como programador, uno tiende a quedarse mirando el código por largos periodos de tiempo, sobre todo cuando queremos encontrar un error. A esta actividad de encontrar un problema o un *bug* (🐛) en un programa se le conoce como *debugging*. Si nuestro código es ordenado y leíble, entonces nos puede ahorrar tiempo en encontrar *bugs*.

3. Se escribe código más rápido

A veces, puede tomar tiempo decidir cuál es el nombre más apropiado para cada elemento (variables, funciones y clases). El tener reglas definidas para nombrar objetos nos ayuda a tomar menos decisiones, y por lo tanto, nombrar y escribir más rápido.

4. Nombres de archivos y variables predecibles

En proyectos grandes, lo más seguro es que se usarán múltiples archivos, clases y funciones escritas por ti o por otra persona. Al tener que reutilizar dicho código, es difícil recordar el nombre de cada función, método y clase escrito, lo que provoca perder tiempo buscando dichos nombres manualmente. El tener reglas para escribir y nombrar archivos y variables nos puede ayudar en ahorrar tiempo en esa búsqueda.

Guía de estilo en Python: PEP8

PEP8 es la guía de estilo propuesta por la documentación oficial de Python. Esta guía define muchas reglas que siguen los pilares de la filosofía de los creadores de Python. Puedes encontrar la especificación completa aquí.

En IIC2233 utilizaremos solo un subconjunto de reglas de PEP8 para escribir nuestro código (y esperamos que los estudiantes también lo usen en sus tareas). Estas reglas son las siguientes:

- Aplicar `snake_case` para nombres de variables, atributos, funciones y métodos.

`snake_case` es la práctica de escribir palabras o frases compuestas usando `_` para separar los distintos elementos del nombre y sin espacios. Existen variantes que utilizan mayúsculas para la primera letra, pero en IIC2233 solo usamos minúsculas para este tipo de nombres, y lo usaremos para nombrar **prácticamente todo**, menos los nombres de clases. Esto es: **nombres de variables**, **atributos**, **funciones** y **métodos**.

- Aplicar `CamelCase` para nombres de clases

Por el otro lado, `CamelCase` es la práctica de escribir palabras o frases compuestas sin espacios ni separador entre elementos, y por cada elemento se comienza con una mayúscula. Existen variantes donde la primera letra utilizan minúscula (*lower cammel case*), pero en IIC2233 utilizaremos la opuesta: *upper cammel case*. Este tipo de escritura lo reservamos para el **nombre de clases**.

A continuación, un ejemplo de escritura siguiendo `snake_case` y `CamelCase`:

```
class PerroAmigable:

    def __init__(self, nombre, edad):
        self.nombre = nombre
        self.edad = edad

    def saludar_amigo(self, amigo):
```

```
        print("¡Woof!")
        print("¡Hola " + amigo.nombre + "!")


def saludo_mutuo(perro_1, perro_2):
    perro_1.saludar_amigo(perro_2)
    perro_2.saludar_amigo(perro_1)
    print()


perro_juan = PerroAmigable("Juan", 21)
perro_pedro = PerroAmigable("Pedro", 20)
perro_jose = PerroAmigable("Jose", 22)

saludo_mutuo(perro_juan, perro_pedro)
saludo_mutuo(perro_jose, perro_pedro)
saludo_mutuo(perro_juan, perro_jose)
```

- Nombres de variables descriptivos.

Es usual que al programar se nos ocurra nombres de variables simples como `a` o `variable`. Si bien tal vez esto ahorra en tiempo de escritura, esto no genera código leíble o fácil de entender. Al volver a leer código como el siguiente, nos produce dudas de que realmente hace lo que estamos leyendo:

```
def funcion(l):
    i = 0
    a = l[0]
    for j in range(1, len(l)):
        var = l[j]
        if var > a:
            j = i
            a = var
    return i
```

En cambio, al utilizar nombres más descriptivos, queda más claro lo que hace la función:

```
def indice_de_maximo(lista):
    indice_actual = 0
    maximo_actual = lista[0]
    for indice in range(1, len(lista)):
        elemento = lista[indice]
        if elemento > maximo_actual:
            indice_actual = indice
            maximo_actual = elemento
    return indice_actual
```

- *Imports* al comienzo del archivo.

Esta convención nos permite ordenar en secciones cada archivo y nos permite encontrar todas las dependencias externas a otras librerías o módulos en un solo lugar, en vez de múltiples secciones:

```
import os
import random

# ...
# El resto del código
```

Más detalles de las sentencias de importación se detallan en el apunte sobre **modularización**.

- Un espacio después de “,”.

Para mantener orden, formato y legibilidad, esta regla se aplica en todos los casos que se usan comas al enumerar elementos:

```
from random import randint, choice  # Espacios al importar


def operaciones_con_potencia(lista, potencia):  # Espacios en
definición de funciones
    suma = 0
    multiplicacion = 1
    for elemento in lista:
        suma += elemento ** potencia
        multiplicacion *= elemento ** potencia
    return suma, multiplicacion  # Espacios en retornos


numeros = [3, 6, 2, 4]  # Espacios en definición de listas
operaciones_con_potencia(numeros, 2)  # Espacios en llamadas de
funciones
```

- Espacios a cada lado de operadores.

Para mantener orden, formato y legibilidad, esta regla se aplica siempre que se haga uso de operadores matemáticos:

```
valor_1 = ((6 ** 17) - 1) / 2
resultado = (1 + valor_1) * 3
resto_en_dos = resultado % 2
```

- No usar *tabs*. Usar 4 espacios para indentar.

Python 3 no permite mezclar ambos tipos en un archivo para indentar bloques de código. Se prefiere usar **cuatro espacios** para cada nivel de indentación, no *tabs*. Dependiendo del editor de

texto, hay formas de configurar el uso de espacios y *tabs* al escribir automáticamente. Las tres interfaces de desarrollo recomendadas para el curso permiten esta funcionalidad, puedes encontrar manuales de instalación aquí.

- Líneas de largo máximo de 100 caracteres (incluyendo espacios).

Es recurrente el problema de líneas muy largas, sobre todo en líneas ya indentadas. Generalmente, se establece el límite de 80 caracteres como máximo para seguir una convención de compatibilidad con herramientas de edición que establecen este límite y permiten ver el código solo de ese ancho de espacio. Pero, en el curso relajaremos esta regla y trabajaremos con líneas de **máximo 100 caracteres**. Hay varias formas de evitar este problema, pero depende del contexto en el que se encuentre el problema. A veces es conveniente aprovechar la creación de variables extras para expresiones largas y acortar líneas, o en casos extremos, el uso de `\` que permite separar una línea larga en la siguiente.

Por ejemplo, el siguiente código tiene líneas que se pueden acortar:

```
print("Debe ingresar dos números")
valor_1 = int(input("Ingrese primer número"))
valor_2 = int(input("Ingrese segundo número"))
# La siguiente línea es muy larga
if valor_1 + valor_2 >= 1000 and (valor_1 - valor_2 >= 1000 or valor_2
- valor_1 >= 1000):
    print("Los números suman mucho y están muy separados")
elif valor_1 + valor_2 >= 1000:
    print("Los números suman mucho")
elif (valor_1 - valor_2 >= 1000 or valor_2 - valor_1 >= 1000):
    print("Los números están muy separados")
else:
    print("Los números son apropiados")
```

Mediante el uso de `\`:

```
print("Debe ingresar dos números")
valor_1 = int(input("Ingrese primer número"))
valor_2 = int(input("Ingrese segundo número"))
# Se separó usando \
if valor_1 + valor_2 >= 1000 and \
            (valor_1 - valor_2 >= 1000 or valor_2 - valor_1 >= 1000):
    print("Los números suman mucho y están muy separados")
elif valor_1 + valor_2 >= 1000:
    print("Los números suman mucho")
elif (valor_1 - valor_2 >= 1000 or valor_2 - valor_1 >= 1000):
    print("Los números están muy separados")
else:
    print("Los números son apropiados")
```

O el uso de variables auxiliares:

```python
print("Debe ingresar dos números")
valor_1 = int(input("Ingrese primer número"))
valor_2 = int(input("Ingrese segundo número"))
# Variables auxiliares para mejorar legibilidad
suma = valor_1 + valor_2
diferencia = abs(valor_1 - valor_2)
if suma >= 1000 and diferencia >= 1000:
    print("Los números suman mucho y están muy separados")
elif suma >= 1000:
    print("Los números suman mucho")
elif diferencia >= 1000:
    print("Los números están muy separados")
else:
    print("Los números son apropiados")
```

En general, se prefiere reescribir el código de tal forma de evitar usar `\`, ya que no es tan legible. También, la intención de esta regla es no escribir líneas largas en general, no solo arreglar aquellas que superen el límite de 100 caracteres.